NexaAssist

LLMOps System Design

STATUS — Teaching draft **OWNER — Not assigned** **LAST UPDATED — 23 Oct 2026**

Authors	NexaAssist workshop
Reviewers	Engineering + SME (pending)
Related docs	Course map; C4 views; handouts
Scope	Policy RAG training design

1. Abstract

NexaAssist answers staff questions from registered synthetic banking-policy documents. It separates offline knowledge preparation from online retrieval and generation, preserving date scope, citations and answer provenance. This document describes the inspected training implementation and explicitly identifies course concepts that remain future work.

The implementation uses LangChain LCEL, MongoDB Atlas Vector Search, FastAPI and Streamlit. Required labs are offline. Provider quality, Atlas compatibility, Datadog export and production load have not been established by the training fakes. Organizational owners and targets remain unconfirmed.

2. Goals and Non-Goals

Goals	Non-goals
Explain all sixteen lessons with mapped practicals	Copying the course text or replacing MongoDB Atlas
Preserve typed, grounded policy answers	Credit decisions or account actions
Make quality and SLO evidence reviewable	Claiming live performance from fakes
Rehearse a compatible rollback decision	Automatic index promotion or production deployment

3. Background and Problem Statement

The Educative course develops an internal support assistant and progressively adds ingestion, retrieval, prompt management, evaluation, serving, conversation state and feedback. NexaAssist adapts the same responsibilities to synthetic policy documents. The first decks underrepresented several course lessons; the revised decks now map all sixteen lessons to specific sections and exercises.

The invariant is that the UI renders, routers validate and delegate, services coordinate workflows, chains contain pure LangChain logic, and database modules own MongoDB access. No loan approval, account action or autonomous tool execution is authorized. External service configuration does not establish measured availability or data permission.[1]

1RAG retrieves source evidence before generation; grounding and source access remain application responsibilities.

4. Proposed Architecture

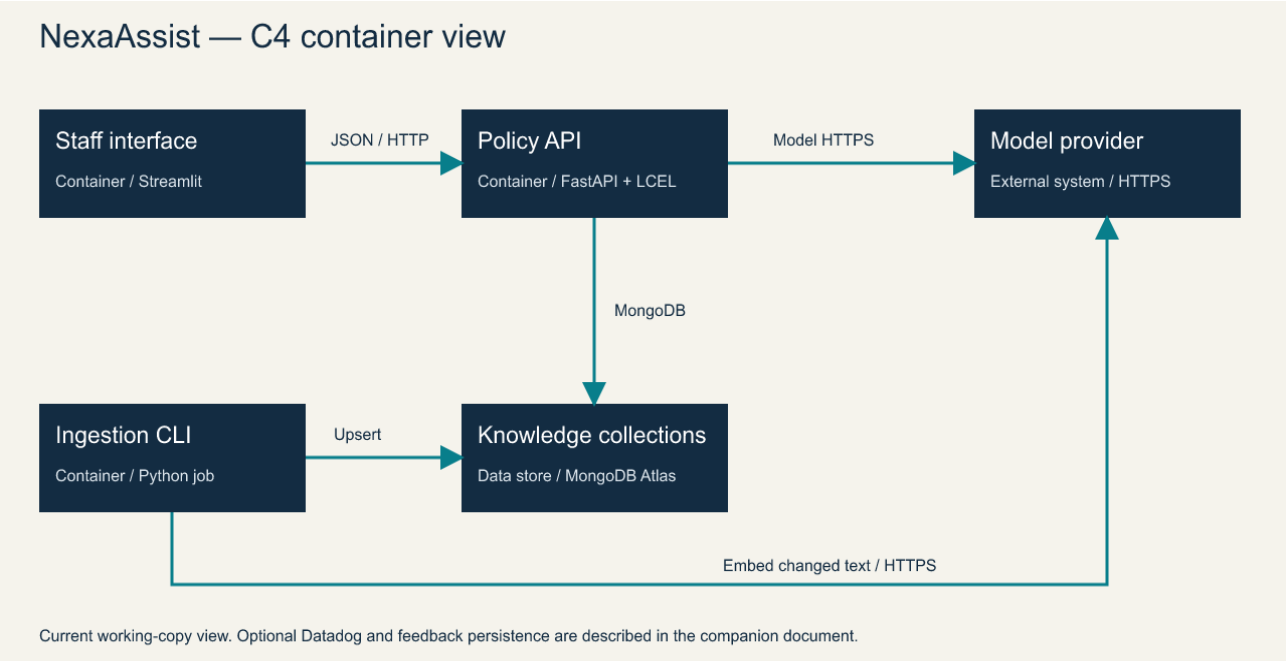


Figure 1. NexaAssist C4 container view; optional telemetry and feedback are described below.

Core components

Component	Responsibility	Primary storage	Failure behavior
Ingestion CLI	Verify, parse, redact and upsert changed documents	Registered files; Atlas chunks/parents	Isolate failed documents and report failure
Retrieval service	Apply query and corpus/date scope	Atlas Vector Search + parent store	Propagate dependency error; report empty evidence
Answer chain	Render prompt, generate and check draft	Model API and bounded context	Abstain or clarify; one citation repair
Feedback workflow	Save stable submissions and reviewer decisions	MongoDB feedback collection	Reject identity conflict; surface unavailable store
Observability/ evaluation	Record operational and semantic evidence	Local reports; optional Datadog	Preserve answer when optional publication fails

5. Request Lifecycle

a. Streamlit submits a question, policy date and optional product to POST /answer.
b. Validate the contract and quota, then mask selected query identifiers. End-user authorization remains a deployment decision.
c. Load configured retrieval/model settings; apply corpus/date filters and retrieve parents plus amendments.
d. Budget evidence, render the tracked prompt and request a strict draft; abstain if required evidence is missing.
e. No conversation write precedes generation. Feedback is stored separately; multi-turn history is not implemented.
f. Bound provider calls, allow one citation repair and discard detected sensitive output. Readiness bounds its database ping.
g. Return answer, citations, provenance and timings. Optional background judging and telemetry support diagnosis.

6. API and Data Contracts

Primary data contract

Field	Type	Required	Description
query	string	Yes	Bounded non-empty policy question
as_of	date	Yes	Policy lookup date; distinct from loan sanction date
product_id	string or null	No	Narrows the requested product scope
status	enum	Yes, response	answered, insufficient_evidence or needs_clarification
answer	string	Yes, response	Validated explanation, fixed clarification or abstention
citations	array	Yes, response	Exact quotes with matched-child identities
provenance	object or null	Response field	Prompt fingerprint, model/retrieval settings and evidence references

Contract guarantees

a. Requests and drafts must satisfy their typed contracts before a supported answer is returned. Exact quotes must occur in the supplied source context.

b. Ingestion uses deterministic chunk identity and checksums. Feedback IDs derive from institution, submitter and submission identifiers; conflicts are rejected.
c. Capture prompt ID/version/hash, chat and embedding identities, retrieval mode and evidence references. A reproducible release also needs corpus, dataset and container identities.
d. A client-submitted feedback response is investigation context, not server-authenticated proof or a golden reference. A human verifies any correction against policy evidence.

The running local API publishes its schema at

http://127.0.0.1:8000/openapi.json. Update the versioned models with each contract release.

7. Consistency, Idempotency, and Replay

Repeated unchanged ingestion skips re-embedding, while embedding or metadata changes invalidate checksums. Per-document failure isolation permits other documents to proceed and reports failure. There is no atomic blue/green corpus promotion. Parent and chunk stores must remain compatible. Feedback submission retries use stable identity and reject conflicting reuse.

Scenario	Expected behavior	Reasoning
Unchanged ingestion	No re-embedding for matching checksums	Stable identifiers and compatibility metadata
Dependency unavailable	Safe API error; private details hidden	No fabricated answer on dependency failure
Invalid citation or output PII	One citation repair, or discard sensitive draft	Preserve source and output integrity
Changed model/index configuration	Rebuild/evaluate compatible candidates	Equal dimensions alone do not prove compatibility

8. Security and Privacy Considerations

a. Reviewer listing/review uses a shared bearer credential in this training app. Request metadata scopes retrieval. End-user identity, authorization, tenancy and externally exposed deployment boundaries need approval before a real rollout.
b. Corpus and query patterns cover selected email, phone, PAN and Aadhaar identifiers. The output rail checks generated text and quotes. Names, addresses and arbitrary sensitive text are not comprehensively detected. Avoid recording private raw content in ordinary logs.
c. Configuration uses pydantic-settings and SecretStr. Inject secrets at runtime; the application image must exclude .env. Trace payload collection needs a reviewed retention and access policy.
d. Required teaching labs use fakes and temporary copies. Ingestion is an operator CLI. Live evaluation, re-indexing and deployment must use approved credentials, data permissions and budgets.

e. Freshness, retention, deletion, residency and telemetry access are owner decisions. Synthetic fixtures do not authorize processing real customer data. Document those decisions in the use-case manifest before a production release.

9. Operational Readiness

Signal	SLO or alert	Owner	Launch gate
Quality	gates.json placeholders; confirm against reviewed cases	Unassigned	Required
Latency	Representative p95; no live claim	Unassigned	Required
Errors/cost	Measured request mix and approved prices	Unassigned	Required
Artifact drift	Track prompt/model/corpus/dataset identity	Unassigned	Required
Recovery	Rehearse known-good configuration and replay	Unassigned	Required

Rollout constraint: hold while required measurements or approvals are missing. Blue/green corpus promotion and multi-turn memory remain design work.

10. Alternatives Considered

Alternative	Why considered	Why not selected here
PostgreSQL / pgvector	Source-course converged database example	Project requires MongoDB Atlas Vector Search
LiteLLM / Jinja registry	Source-course provider/template abstraction	Existing LangChain integrations and prompt artifacts are sufficient for this layer
Stateful memory and rewriting	Course lesson 14 conversational workflow	Needs separate scope, authorization and multi-turn evaluation
Self-hosted inference	Potential control and steady-load economics	No measured traffic or capacity evidence justifies operating GPUs

11. Open Questions

a. Who owns approval of quality/SLO thresholds and the live evaluation dataset?
b. What end-user identity, data authorization, retention and provider-disclosure rules apply to a real corpus?
c. Which deployment environment, traffic profile and recovery objective will be measured?
d. How should reviewer identity, live prompt-release evaluation, CI and isolated index promotion be approved and verified?

12. Decision and Next Steps

Use the source-aligned workshop as an implementation and review guide. Preserve the working layer boundaries, run the required offline labs, then collect live retrieval/quality and staging traffic evidence in an approved environment. Production remains a hold until required measurements, ownership and rollback rehearsal are complete.

Milestone	Deliverable	Exit criteria
M1	Source-aligned decks, diagrams and offline practicals	Coverage and artifact checks pass
M2	Approved live retrieval/answer evaluation	Measured report against confirmed targets
M3	Staging traffic and recovery rehearsal	SLOs observed; owners and rollback identified
M4	Production release decision	Required evidence and approval complete